Regular Expressions

Greg Ridgeway Ruth Moyer

2026-09-24

Table of contents

1 Introduction **1**

2 Finding patterns in text with `grep()` **2**
2.1 Find letters and numbers . . . 3
2.2 More symbols that help with grepping . . . 7
2.2.1 Carets `^` . . . 7
2.2.2 Dollar Signs `$` . . . 8
2.2.3 Plus Sign `+` . . . 8
2.2.4 Parentheses `()` . . . 9
2.2.5 Vertical Bar `|` . . . 9
2.2.6 Question Mark `?` . . . 11
2.2.7 Boundaries `\b` and `\B` . . . 12

3 Finding and replacing patterns in text with `gsub()` **13**
3.1 Basic find and replace examples . . . 13
3.2 Backreferences . . . 15
3.3 Lookaheads . . . 17

4 Introduction to Web Scraping: A Practical Application of Regular Expressions **20**
4.1 The 10,000 most common English words . . . 20

5 Solutions to the exercises **22**

1 Introduction

A regular expression is a sequence of characters that defines a search pattern. Sometimes we use regular expressions to help us find a pattern in text that we want, like the Find functionality

in Word. Other times, we use a regular expression to help us find and *replace* a piece of text, like the Find and Replace functionality in Word. The two main R functions that we will learn about are `grep()` and `gsub()`. You may use them in verb form (e.g., "I need to grep all the gun cases," "I was grepping like crazy to find all the opioid cases," "I first had to gsub the commas with semicolons"). Regular expressions are available in numerous other software packages. Most of the concepts you learn here will transfer to regular expressions in Linux, Stata, Python, Java, ArcGIS, and many others, although the exact syntax and supported features can vary.

We have already had a brief introduction to regular expressions. When we wanted to select protests in Philadelphia, we ran

```
dataProtest |>
  filter(grepl("Philadelphia, PA", Location))
```

Later, when we work with NIBRS, we will use a regular expression to find agencies in Pennsylvania by looking for "PA" at the start of their ORI code inside an SQL query.

```
initExtension(con, "regexp")

dbGetQuery(con, "
  SELECT DISTINCT ori
  FROM   batch_header
  WHERE  ori REGEXP '^PA'
  LIMIT  10")
```

In the next set of notes, we will work with NCVS data and use a regular expression to select crime labels containing "rape" or "sex."

```
dataExt |>
  filter(grepl("rape|[Ss]ex", V4529)) |>
  distinct(V4529)
```

The examples involving NCVS and NIBRS are previews. The syntax used in all three examples will become clear in these notes.

2 Finding patterns in text with `grep()`

First we will learn about `grep()`, which was first developed in the early 1970s. `grep()` searches data for lines that match a given expression. The name `grep` comes from the `ed` command `g/re/p`, meaning "global regular expression print." Some tangible examples will help us see how `grep()` works.

Run the following line that provides a collection of text elements with a diverse range of capitalization, letters, punctuation, and other features. We will use this for practice and testing.

```
dataText <- c("3718 Locust Walk",
              "4 Privet Drive",
              "10880 Malibu Point",
              "221B Baker St.",
              "19104-6286",
              "20015",
              "90291",
              "90210",
              "(215) 573-9097",
              "215-573-9097",
              "2155739097",
              "Kendall Roy",
              "Roman Roy",
              "Siobhan Roy",
              "Roy Wood, Jr.",
              "Royal Caribbean",
              "Casino Royale",
              "two children",
              "2 children",
              "twins",
              "Philadelphia",
              "Philly",
              "Phila",
              "Dr. Phil",
              "$23456",
              "$10000",
              "$60,000")
```

2.1 Find letters and numbers

Let's find all of the items that have the letter "a" in `dataText`.

```
grep("a", dataText)
```

```
 [1]  1  3  4 12 13 14 16 17 21 23
```

This shows the indices of which elements of `dataText` have an "a" in them. Sure enough the first element, "3718 Locust Walk", has an "a" and you can confirm that the other indices `grep()` returned also indicate elements of `dataText` that have an "a".

For most of these notes we are going to set `value=TRUE`, which will return the actual elements matched rather than their indices. This will make it easier to check that `grep()` is finding the elements that we are expecting to match.

```
grep("a", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "221B Baker St."
 [4] "Kendall Roy"        "Roman Roy"          "Siobhan Roy"
 [7] "Royal Caribbean"    "Casino Royale"      "Philadelphia"
[10] "Phila"
```

As you can see, `grep()` uses the following general syntax: grep("what we're searching for", the text, and an optional `value=TRUE` if we want to return text). If we put `value=FALSE` (which is the default if we do not set `value`), we will just receive the index in the text where we can find what we are searching for.

Let's try another example. Instead of a letter, let's try to find a number such as "1". Specifically, which items in `dataText` have a "1" in them?

```
grep("1", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "221B Baker St."
 [4] "19104-6286"         "20015"              "90291"
 [7] "90210"              "(215) 573-9097"     "215-573-9097"
[10] "2155739097"         "$10000"
```

We can also search for any one of several characters. We place the possible characters within square brackets `[ ]`. For example, let's find items that contain numbers.

```
grep("[0123456789]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "2 children"
[13] "$23456"             "$10000"             "$60,000"
```

If we wanted items that contain an odd digit, we could do this.

```
grep("[13579]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "221B Baker St."
 [4] "19104-6286"         "20015"              "90291"
 [7] "90210"              "(215) 573-9097"     "215-573-9097"
[10] "2155739097"         "$23456"             "$10000"
```

So the `[]` in a regular expression means "match any of these characters." We can also place square brackets next to each other. For example, let's say we wanted to find an item in `dataText` that has four adjacent numbers.

```
grep("[0-9][0-9][0-9][0-9]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "19104-6286"
 [4] "20015"              "90291"              "90210"
 [7] "(215) 573-9097"     "215-573-9097"       "2155739097"
[10] "$23456"             "$10000"
```

Note that we can use the shorthand `0-9` to mean any number between 0 and 9, including 0 and 9. This regular expression says "find a number, followed by a number, followed by another number, followed by another number." Alternatively we can use `{4}` to mean "match the preceding item exactly four times."

```
grep("[0-9]{4}", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "19104-6286"
 [4] "20015"              "90291"              "90210"
 [7] "(215) 573-9097"     "215-573-9097"       "2155739097"
[10] "$23456"             "$10000"
```

`{n}` means the preceding item will be matched exactly n times. Note that this also matches text that has five or more numbers in a row, since if they have five numbers in a row, then they will also have four numbers in a row. Later we will learn about how to find exactly four numbers in a row.

It is not used often, but we can also use squiggly or curly braces to make a range.

```
grep("[0-9]{5,10}", dataText, value=TRUE)
```

```
[1] "10880 Malibu Point" "19104-6286"         "20015"
[4] "90291"              "90210"              "2155739097"
[7] "$23456"             "$10000"
```

Thus `{n,m}` will find something matched between n and m times. We have used examples above with numbers, but you can apply this syntax to letters.

```
grep("[a-zA-Z]{5}", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "Kendall Roy"        "Roman Roy"
 [7] "Siobhan Roy"        "Royal Caribbean"    "Casino Royale"
[10] "two children"       "2 children"         "twins"
[13] "Philadelphia"       "Philly"             "Phila"
```

Note how we used the shorthand `a-z` to mean any lowercase letter and `A-Z` to mean any uppercase letter. `grep()` will match a different set of elements if we just search for lowercase letters.

```
grep("[a-z]{5}", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "Kendall Roy"        "Siobhan Roy"        "Royal Caribbean"
 [7] "Casino Royale"      "two children"       "2 children"
[10] "twins"              "Philadelphia"       "Philly"
```

Searches using `grep()` are case-sensitive. For example, let's find all items that contain capital letters.

```
grep("[A-Z]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "Kendall Roy"        "Roman Roy"
 [7] "Siobhan Roy"        "Roy Wood, Jr."      "Royal Caribbean"
[10] "Casino Royale"      "Philadelphia"       "Philly"
[13] "Phila"              "Dr. Phil"
```

Exercises

1. Find text with vowels
2. Find text with a number immediately followed by a letter
3. What is the difference between `grep("Roy", dataText, value=TRUE)` and `grep("[Roy]", dataText, value=TRUE)`?

2.2 More symbols that help with grepping

So far you have seen that we use `[]` to match any character listed between the `[]`, the `-` to specify a range to match like `0-9`, `a-z`, and `A-Z`, and the `{}` to match the preceding item multiple times.

In addition to these, there are several more symbols or sequences of symbols that are useful. The number of symbols on the keyboard is fairly limited compared to the numerous combinations of patterns of text we might wish to find. As a result you will see that some of these symbols are used in very different ways depending on the context.

2.2.1 Carets ^

Let's look at the caret ^ symbol. When we put a ^ within square brackets, it means "not". Let's try to find text in `dataText` that has something that is not a letter or a space immediately followed by a letter.

```
grep("[^A-Za-z ][A-Za-z]", dataText, value=TRUE)
```

```
[1] "221B Baker St."
```

"221B Baker St." has a character that is not a letter (1) immediately followed by a letter (B).

Outside square brackets, the ^ means something *completely* different. It means "the beginning of the text". For example, the following regular expression matches text where the first character is either an uppercase or a lowercase letter.

```
grep("^[A-Za-z]", dataText, value=TRUE)
```

```
 [1] "Kendall Roy"     "Roman Roy"       "Siobhan Roy"     "Roy Wood, Jr."
 [5] "Royal Caribbean" "Casino Royale"   "two children"    "twins"
 [9] "Philadelphia"    "Philly"          "Phila"           "Dr. Phil"
```

2.2.2 Dollar Signs $

While we use carets to signal the beginning of the text, the dollar sign signals the end of the text. For example, to search for all items that end with either an uppercase or lowercase letter, we would do the following:

```
grep("[A-Za-z]$", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "Kendall Roy"        "Roman Roy"          "Siobhan Roy"
 [7] "Royal Caribbean"    "Casino Royale"      "two children"
[10] "2 children"         "twins"              "Philadelphia"
[13] "Philly"             "Phila"              "Dr. Phil"
```

Note how all of these have a letter as the last character. We can be more specific and ask for text that ends with the letter "y" or end with a 7.

```
grep("y$", dataText, value=TRUE)
```

```
[1] "Kendall Roy" "Roman Roy"   "Siobhan Roy" "Philly"
```

```
grep("7$", dataText, value=TRUE)
```

```
[1] "(215) 573-9097" "215-573-9097"   "2155739097"
```

2.2.3 Plus Sign +

The + means "match the preceding item one or more times." For example, suppose we wanted to find items in our list that have numbers and those numbers are followed by letters.

```
grep("[0-9]+[A-Za-z]+", dataText, value=TRUE)
```

```
[1] "221B Baker St."
```

Or search for text that starts with some numbers, then has a space, followed by some letters, and then ends.

```
grep("^[0-9]+ [A-Za-z]+$", dataText, value=TRUE)
```

```
[1] "2 children"
```

> Exercises
>
> 4. Find the Roy children. That is, pick out Kendall, Roman, and Siobhan Roy.
> 5. Find text that starts with a number and ends with a letter. (Hint: You will probably need to use `^`, `$`, and `+`).

2.2.4 Parentheses `()`

Parentheses group part of a regular expression into a single unit. This becomes especially useful with alternatives and optional text. For example, suppose we wanted to find the word "two".

```
grep("(two)", dataText, value=TRUE)
```

```
[1] "two children"
```

On its own the parentheses are no different than just running the regular expression "two" with no parentheses. However, in combination with `|` and `?` it gets more interesting and powerful.

2.2.5 Vertical Bar `|`

The vertical bar functions as an "or." Suppose we wanted to get both the phrases "two children" and "2 children", allowing the number to be spelled out or not. We would use parentheses as well as a vertical bar.

```
grep("(two|2) children", dataText, value=TRUE)
```

```
[1] "two children" "2 children"
```

Let's find all Los Angeles and Washington DC ZIP codes.

```
grep("(902|200)[0-9][0-9]", dataText, value=TRUE)
```

```
[1] "20015" "90291" "90210"
```

Here's how we can find four-letter words that are separated by spaces or occur at the beginning or end of the text.

```
grep("( |^)[A-Za-z]{4}( |$)", dataText, value=TRUE)
```

```
[1] "3718 Locust Walk" "Dr. Phil"
```

`( |^)` says that we start with either a space or the beginning of the text. `( |$)` says that at the end there must be a space or the end of the text. In the middle there has to be exactly four letters. Note that this did not select "Phil." since it has a period following it. We'll need a more general regular expression to select that one as well.

Let's try to find phone numbers in `dataText`. The problem is that phone numbers have three different formats in our data. The easiest phone number pattern to find is the one with 10 digits in a row.

```
grep("[0-9]{10}", dataText, value=TRUE)
```

```
[1] "2155739097"
```

It is also not too hard to find the hyphenated phone number pattern.

```
grep("[0-9]{3}-[0-9]{3}-[0-9]{4}", dataText, value=TRUE)
```

```
[1] "215-573-9097"
```

The phone number format with parentheses needs a little more caution. Parentheses have special meaning in regular expressions. Several other characters can also have special meanings, including `\ ^ $ { } [ ] ( ) . * + ? |`. A hyphen has special meaning inside square brackets, where it can define a range. To search for a special character literally, we usually place a backslash before it. However, the backslash is also special in an R string, so it must itself be preceded by another backslash. To look for phone numbers with parentheses, we use a regular expression like this.

```
grep("\\([0-9]{3}\\) [0-9]{3}-[0-9]{4}", dataText, value=TRUE)
```

```
[1] "(215) 573-9097"
```

There is a workaround to the extra backslashes, which can be overwhelming in some regular expressions. An R raw string literal prevents R from interpreting the backslashes as special characters. Wrap your text inside `r"( ... )"` and you can do away with double backslashes.

```
# an R string literal requiring only single backslashes
r"(\([0-9]{3}\) [0-9]{3}-[0-9]{4})"
```

```
[1] "\\([0-9]{3}\\) [0-9]{3}-[0-9]{4}"
```

```
# regular expression, now with 50% fewer backslashes!
grep(r"(\([0-9]{3}\) [0-9]{3}-[0-9]{4})", dataText, value=TRUE)
```

```
[1] "(215) 573-9097"
```

Now, let's put all of these patterns together and use `|` to search for any phone number in any of the three formats.

```
grep("[0-9]{10}|[0-9]{3}-[0-9]{3}-[0-9]{4}|\\([0-9]{3}\\) [0-9]{3}-[0-9]{4}",
     dataText, value=TRUE)
```

```
[1] "(215) 573-9097" "215-573-9097"   "2155739097"
```

2.2.6 Question Mark ?

The question mark indicates optional text. To illustrate,

```
grep("Phil(adelphia)", dataText, value=TRUE)
```

```
[1] "Philadelphia"
```

```
grep("Phil(adelphia)?", dataText, value=TRUE)
```

```
[1] "Philadelphia" "Philly"       "Phila"        "Dr. Phil"
```

The first one is no different from searching for the word "Philadelphia," but the second one says that the "adelphia" part is optional. Note that this regular expression also picked up "Dr. Phil". Again we will need a more careful regular expression to avoid matching this name and only select abbreviations and nicknames for Philadelphia.

2.2.7 Boundaries \b and \B

\b matches a position at the edge of a word, such as where a word meets a space or punctuation, or at the beginning or end of the text. So another way to find all text with four-letter words, including "Phil.", is

```
grep("\\b[A-Za-z]{4}\\b", dataText, value=TRUE)
```

```
[1] "3718 Locust Walk" "Roy Wood, Jr."    "Dr. Phil"
```

Remember, the \ is a special character for R. To "protect" the backslash we put another backslash in front of it. If we did not include the \b in this regular expression, then we would have also matched words with five or more letters too.

For several backslash shortcuts, the uppercase version means "not" the lowercase version. \B matches a position that is not at the edge of a word. Similarly, \w matches a word character, while \W matches a character that is not a word character. We can use \B to find Phil only when more word characters follow it.

```
# Phil followed by something that is not a word boundary
grep("Phil\\B", dataText, value=TRUE)
```

```
[1] "Philadelphia" "Philly"       "Phila"
```

Exercises

Write regular expressions to find the following items in dataText.

6. Find commas
7. Find ZIP codes
8. Find text containing a six-letter word
9. Find mentions of Philadelphia
10. Find capitalized words
11. Find the addresses
12. Find a shorter way to get phone numbers using ?

3 Finding and replacing patterns in text with `gsub()`

`gsub()` stands for "global substitution." It is useful for automating the editing of text, including deleting characters from text or extracting only parts of text.

3.1 Basic find and replace examples

Let's remove all numbers from our `dataText` list.

```
gsub("[0-9]","",dataText)
```

```
 [1] " Locust Walk"    " Privet Drive"   " Malibu Point"   "B Baker St."
 [5] "-"               ""                ""                ""
 [9] "() -"            "--"              ""                "Kendall Roy"
[13] "Roman Roy"       "Siobhan Roy"     "Roy Wood, Jr."   "Royal Caribbean"
[17] "Casino Royale"   "two children"    " children"       "twins"
[21] "Philadelphia"    "Philly"          "Phila"           "Dr. Phil"
[25] "$"               "$"               "$,"
```

or remove all the Roy last names

```
gsub(" Roy$","",dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall"
[13] "Roman"              "Siobhan"            "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

or turn selected punctuation to "X"

```
gsub("[.)($-]","X", dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker StX"     "19104X6286"         "20015"
 [7] "90291"              "90210"              "X215X 573X9097"
[10] "215X573X9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, JrX"
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "DrX Phil"
[25] "X23456"             "X10000"             "X60,000"
```

or remove HTML tags from text (very very handy!!!).

```
gsub("<[^>]+>", "",
     "<td><b><a href=\"/movie/Dont-Breathe-2-(2021)#tab=box-office\">Don't Breathe 2</a></b>
```

```
[1] "Don't Breathe 2"
```

This last one we will use a lot. The regular expression starts with `<`, matches one or more characters other than `>`, and ends with `>`. Anything matching this pattern, like `<td>`, will get removed.

The first argument of the `gsub()` function is the "find pattern," what we are asking `gsub()` to find. The second argument is a "replacement pattern" that will replace whatever `gsub()` matched with the find pattern. Lastly, like `grep()`, we need to tell `gsub()` the name of the data object in which it should look for the pattern.

With `gsub()` we will be using a lot of the same grammar, such as carets, brackets, and dashes, that we used with `grep()`. To illustrate, let's remove anything that is not a number from text.

```
gsub("[^0-9]","",dataText)
```

```
 [1] "3718"       "4"          "10880"      "221"        "191046286"
 [6] "20015"      "90291"      "90210"      "2155739097" "2155739097"
[11] "2155739097" ""           ""           ""           ""
[16] ""           ""           ""           "2"          ""
[21] ""           ""           ""           ""           "23456"
[26] "10000"      "60000"
```

Exercise

13. Remove all the commas. Note that we probably just want to remove them from the numbers, but we do not quite yet have the tools to avoid removing the comma from Roy Wood, Jr.

3.2 Backreferences

A slightly more complicated aspect of `gsub()` is using backreferences. Parentheses form a capturing group that saves the text matched by that part of the pattern. You can use multiple pairs of parentheses to capture different parts of the matched text. In the replacement pattern, `\\1` refers to the text captured by the first group, `\\2` refers to the text captured by the second group, and so on.

For example, let's delete the "plus four" part of the ZIP codes in our data.

```
gsub("([0-9]{5})-[0-9]{4}","\\1",dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104"              "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

We wrapped parentheses around the first five digits, so the first capturing group contains only those digits. The replacement pattern replaces everything matched by the find pattern, including the five digits, hyphen, and plus-four digits, with the text captured by the first group.

Let's use `gsub()` to just keep the first two "words" in each element of `dataText`.

```
gsub("^([^ ]+ [^ ]+) .*$", "\\1", dataText)
```

```
 [1] "3718 Locust"     "4 Privet"        "10880 Malibu"    "221B Baker"
 [5] "19104-6286"      "20015"           "90291"           "90210"
 [9] "(215) 573-9097"  "215-573-9097"    "2155739097"      "Kendall Roy"
[13] "Roman Roy"       "Siobhan Roy"     "Roy Wood,"       "Royal Caribbean"
```

```
[17] "Casino Royale"   "two children"    "2 children"      "twins"
[21] "Philadelphia"    "Philly"          "Phila"           "Dr. Phil"
[25] "$23456"          "$10000"          "$60,000"
```

This regular expression says: start at the beginning of the text, find a bunch of characters that are not spaces (remember that the `+` matches the preceding item one or more times), then find a space, then find another bunch of characters that are not spaces, followed by another space, followed by anything else until the end of the text. The `.` is a wildcard that matches any one character. The `*` is like the `+` but it matches the preceding item *zero* or more times. Note how we have used the parentheses. They are wrapped around those first two words. The first capturing group saves those words, and the replacement pattern recalls the captured text with `\\1`.

An alternative strategy is to use `\w`, which matches a "word character," meaning a letter, number, or underscore, and `\s`, which matches a whitespace character, such as a space, tab, or line break. The `+` following `\w` means to match one or more word characters.

```
gsub("^(\\w+\\s\\w+)\\s.*$","\\1",dataText)
```

```
 [1] "3718 Locust"     "4 Privet"        "10880 Malibu"    "221B Baker"
 [5] "19104-6286"      "20015"           "90291"           "90210"
 [9] "(215) 573-9097"  "215-573-9097"    "2155739097"      "Kendall Roy"
[13] "Roman Roy"       "Siobhan Roy"     "Roy Wood, Jr."   "Royal Caribbean"
[17] "Casino Royale"   "two children"    "2 children"      "twins"
[21] "Philadelphia"    "Philly"          "Phila"           "Dr. Phil"
[25] "$23456"          "$10000"          "$60,000"
```

When working with `grep()`, we wrote a regular expression to find all the phone numbers in all the various formats. Now let's use `gsub()` to standardize all the phone numbers to have the hyphenated format, like 123-456-7890. We use the same regular expression you figured out in the exercises when using `?` in `grep()` to find phone numbers, but we insert pairs of parentheses to capture the phone number digits.

```
gsub("^\\(?([0-9]{3})(\\) |-)?([0-9]{3})-?([0-9]{4})","\\1-\\3-\\4",dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "215-573-9097"
[10] "215-573-9097"       "215-573-9097"       "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
```

```
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

Note that the second capturing group contains whatever text matches the optional pattern `(\\) |-)`. We do not need that text in the result, so the replacement pattern does not include `\\2`.

> **Exercise**
>
> 14. Add commas to these numbers `c("9453","2332","4645","1234")`. That is, make these numbers look like 9,453 and 2,332 and so on. Fill in the find pattern and replacement pattern in `gsub("", "", c("9453","2332","4645","1234"))`

3.3 Lookaheads

Although not as commonly used as a backreference, a "lookahead" can be helpful to find what comes next or, more generally, to `gsub()` items that are a bit more complicated. Let's say you wanted to check that every q is followed by a u. If it's not, then insert the u after the q.

Maybe you would do something like this

```
gsub("q[^u]","qu",c("quiet","quick","quagmire","qixotic"))
```

```
[1] "quiet"    "quick"    "quagmire" "quxotic"
```

As you can see, it doesn't quite do what we wanted. It problematically drops the "i" from "quixotic." Remember that the replacement pattern will overwrite whatever matches the find pattern. This find pattern will match the "qi," the q *and* the adjacent character that is not a u.

A lookahead checks the characters that come next without including them in the match. `(?!u)` is a negative lookahead. The `!` means that the next character must not be `u`. Therefore, `q(?!u)` matches a `q` only when it is not followed by a `u`.

```
gsub("q(?!u)", "qu", c("quiet","quick","quagmire","qixotic"), perl=TRUE)
```

```
[1] "quiet"    "quick"    "quagmire" "quixotic"
```

Now we have "quixotic" spelled properly as `gsub()` looked ahead to check for a "u" but did not consider it part of the match to be replaced. Note that we have set `perl=TRUE`. There is not a single regular expression standard. Lookaheads (and there are lookbehinds too) are not part of the POSIX 1003.2 standard that R uses by default. However, you can ask R to use perl-style regular expressions that do support lookaheads (and lookbehinds) by simply setting `perl=TRUE`.

Let's say we want to add commas to large numbers to display like 1,234,567 instead of 1234567. Ready for this one?

```
gsub("(?<!\\.[0-9]{0,100})([0-9])(?=([0-9]{3})+($|\\.))", "\\1,",
     c("$1234567890","234","1234","12345","12345.6789"),
     perl=TRUE)
```

```
[1] "$1,234,567,890" "234"            "1,234"          "12,345"
[5] "12,345.6789"
```

First note the replacement text, `\\1,`. This reproduces the text captured by the first group, followed by a `,`. The challenge for the matching part is to write a regular expression that identifies those digits where a comma is needed to its right. There are three parts to the regular expression.

- `(?<!\\.[0-9]{0,100})`: This negative lookbehind makes sure there is no decimal point followed by up to 100 digits to the left of the digit. We use `{0,100}` because the unbounded repetition symbol `*` is not allowed in a lookbehind
- `([0-9])`: Match a single digit and save it in the first capturing group
- `(?=([0-9]{3})+($|\\.))`: This is a "lookahead" checking that there are blocks of 3 digits to the right until either the end or a decimal

This gets you a handy regular expression to produce nicer looking numbers. Just so you know, R also provides some handy tools for formatting numbers.

```
prettyNum(c(1234567890, 234, 1234, 12345, 12345.6789), big.mark=",")
```

```
[1] "1,234,567,890" "234"           "1,234"         "12,345"
[5] "12,345.68"
```

Here's how lookaheads are going to be very important for us in working with data. We often get datasets in comma-separated value format, typically with a ".csv" extension on the file name. The R function `read.csv()` can read in data in this format. Problems can occur when values in the dataset have commas inside of them.

Here's some example text that could be problematic.

```
text.w.quotes <- 'white,male,"loitering,narcotics",19,"knife,gun"'
cat(text.w.quotes,"\n")
```

```
white,male,"loitering,narcotics",19,"knife,gun"
```

Some of the commas in this text are separating values for race, sex, arrest charge, age, and recovered contraband. However, there are other commas inside the quotes listing the arrest charges and the contraband items. Fortunately, `read.csv()` is smart enough to keep quoted items together as one data element, as long as the parameter `quote = "\"'"`, which is the default. Other functions are not so kind. Later we will use `dbWriteTable()` to build an SQL database and it will think that all the commas separate the values. So it will think there is a separate `"loitering` data element and then a `narcotics`. And the same for `"knife` and `gun"`.

So here is a very handy regular expression using lookaheads that changes commas that are not inside quotes to semicolons. You can also choose a stranger symbol, like | or @. This regular expression looks for a `,` and then it uses a lookahead for the remainder of the line. It will match that comma if there are no quotes for the rest of the line, `[^\"]`, or if there are only pairs of quotes each with non-quote characters inside of them, `\"[^\"]*\"`.

```
gsub(",(?=([^\"]|\"[^\"]*\")*$)",";",text.w.quotes,perl=TRUE) |>
   cat()
```

```
white;male;"loitering,narcotics";19;"knife,gun"
```

As you can see, commas inside the quotes are preserved and those outside have been transformed into `;`. Now we would be able to tell a function like `dbWriteTable()` that the data elements are separated by `;` and it will read in the data properly.

Exercises

Make the following changes to `dataText`.

15. Change Dr. Phil to Dr. Phillip
16. Spell out Philadelphia
17. Keep just the first word or first number
18. Keep only area codes of phone numbers

4 Introduction to Web Scraping: A Practical Application of Regular Expressions

As you have already seen, regular expressions are an extremely valuable tool when working with data. In fact, we are going to learn about web scraping next. Web scraping enables us to extract data from a website by searching the underlying HTML code for the website and extracting the desired data. To do web scraping, you are going to rely heavily on regular expressions.

4.1 The 10,000 most common English words

Suppose we want a list of the 10,000 most common words in English. wiktionary.org provides this list. Have a look at the webpage.

The `scan()` function in R can read data from a text file, but if given a URL, it will download the HTML code for that page.

```
url <- "https://en.wiktionary.org/wiki/Wiktionary:Frequency_lists/PG/2006/04/1-10000"
html <- scan(url, what="", sep="\n")
```

The number of items may change as Wiktionary updates its headers, footers, and other page elements. Instead of relying on particular line numbers, we will use regular expressions to identify the HTML table cells that contain the data.

The first several elements contain HTML code that sets up the page.

```
html[1:5]
```

```
[1] "<!DOCTYPE html>"
[2] "<html class=\"client-nojs vector-feature-language-in-header-enabled vector-
feature-language-in-main-menu-disabled vector-feature-language-in-main-page-
header-disabled vector-feature-page-tools-pinned-disabled vector-feature-
toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-
feature-limited-width-clientpref-1 vector-feature-limited-width-content-
enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-
pinned-clientpref-1 skin-theme-clientpref-day vector-sticky-header-enabled vector-
toc-available skin-thumbsize-clientpref-standard\" lang=\"en\" dir=\"ltr\">"
[3] "<head>"
[4] "<meta charset=\"UTF-8\">"
[5] "<title>Wiktionary:Frequency lists/PG/2006/04/1-10000 - Wiktionary, the free dictionary<
```

The HTML tag `<td>` marks a cell in a table. The caret in `^<td>` restricts the match to elements that begin with that tag. We can use `grep()` to retain those elements.

```
tableCells <- grep("^<td>", html, value=TRUE)
tableCells[1:9]
```

```
[1] "<td><b>Rank</b></td>"
[2] "<td><b>Word</b></td>"
[3] "<td><b>Count (per billion)</b>"
[4] "<td>1</td>"
[5] "<td><a href=\"/wiki/the\" title=\"the\">the</a></td>"
[6] "<td>56271872"
[7] "<td>2</td>"
[8] "<td><a href=\"/wiki/of\" title=\"of\">of</a></td>"
[9] "<td>33950064"
```

Each element still contains HTML tags. The following regular expression finds a `<`, followed by zero or more characters that are not `>`, followed by a `>`. We use `gsub()` to remove every matching tag.

```
tableCells <- gsub("<[^>]*>", "", tableCells)
tableCells[1:9]
```

```
[1] "Rank"                "Word"                "Count (per billion)"
[4] "1"                   "the"                 "56271872"
[7] "2"                   "of"                  "33950064"
```

The table cells occur in groups of three: rank, word, and count. The first group contains the table headers, so we start with the fifth cell, the first word, and then keep every third cell.

```
words <- tableCells[seq(5, length(tableCells), by=3)]
length(words)
```

```
[1] 10009
```

That is more words than expected. It seems the word "Word" shows up nine times.

```
table(words) |> sort() |> tail()
```

```
words
youthful     zeal  zealous     Zeus     zone     Word
       1        1         1        1        1        9
```

"Word" is inserted on the webpage as a repeating header every 1,000 words. Let's drop those.

```
words <- setdiff(words, "Word")
length(words)
```

```
[1] 10000
```

Now `words` contains the list of 10,000 most common words. Here are the first 50.

```
words[1:50]
```

```
 [1] "the"   "of"    "and"   "to"    "in"    "i"     "that"  "was"   "his"
[10] "he"    "it"    "with"  "is"    "for"   "as"    "had"   "you"   "not"
[19] "be"    "her"   "on"    "at"    "by"    "which" "have"  "or"    "from"
[28] "this"  "him"   "but"   "all"   "she"   "they"  "were"  "my"    "are"
[37] "me"    "one"   "their" "so"    "an"    "said"  "them"  "we"    "who"
[46] "would" "been"  "will"  "no"    "when"
```

Exercises

19. We were told "i before e except after c, or when sounded like a as in neighbor or weigh". Is that true?
20. Find words with punctuation
21. Find words that do not have aeiou in the first four characters

5 Solutions to the exercises

1. Find text with vowels

```
grep("[AEIOUaeiou]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "Kendall Roy"        "Roman Roy"
 [7] "Siobhan Roy"        "Roy Wood, Jr."      "Royal Caribbean"
[10] "Casino Royale"      "two children"       "2 children"
[13] "twins"              "Philadelphia"       "Philly"
[16] "Phila"              "Dr. Phil"
```

2. Find text with a number immediately followed by a letter

```
grep("[0-9][A-Za-z]", dataText,value=TRUE)
```

```
[1] "221B Baker St."
```

3. What is the difference between `grep("Roy", dataText, value=TRUE)` and `grep("[Roy]", dataText, value=TRUE)`? `Roy` matches the sequence "Roy" anywhere in the text, so it matches names such as "Kendall Roy" as well as "Royal Caribbean." `[Roy]` matches any single "R", "o", or "y" character. To match only text that is exactly "Roy", use `^Roy$`.

```
grep("Roy", dataText,value=TRUE)
```

```
[1] "Kendall Roy"     "Roman Roy"       "Siobhan Roy"     "Roy Wood, Jr."
[5] "Royal Caribbean" "Casino Royale"
```

```
grep("[Roy]", dataText,value=TRUE)
```

```
 [1] "3718 Locust Walk"   "10880 Malibu Point" "Kendall Roy"
 [4] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
 [7] "Royal Caribbean"    "Casino Royale"      "two children"
[10] "Philly"
```

4. Find the Roy children

```
grep("Roy$", dataText, value = TRUE)
```

```
[1] "Kendall Roy" "Roman Roy"   "Siobhan Roy"
```

5. Find text that starts with a number and ends with a letter

```
# note that 0-z includes : ; < = > ? @ [ \ ] ^ _ `
grep("^[0-9][0-z ]+[A-Za-z]$", dataText, value=TRUE)
```

```
[1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
[4] "2 children"
```

6. Find commas

```
grep(",", dataText,value=TRUE)
```

```
[1] "Roy Wood, Jr." "$60,000"
```

Remember that not every regular expression is complicated. Sometimes you just need to search for something specific and it requires no fancy regular expression.

7. Find ZIP codes

```
grep("^[0-9]{5}$|^[0-9]{5}-", dataText, value=TRUE)
```

```
[1] "19104-6286" "20015"      "90291"      "90210"
```

or

```
grep("^[0-9]{5}(-[0-9]{4})?$", dataText, value=TRUE)
```

```
[1] "19104-6286" "20015"      "90291"      "90210"
```

8. Find text containing a six-letter word

```
grep("\\b[A-Za-z]{6}\\b", dataText, value=TRUE)
```

```
[1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
[4] "Casino Royale"      "Philly"
```

9. Find mentions of Philadelphia

```
grep("Phil(adelphia|ly|a)", dataText, value=TRUE)
```

```
[1] "Philadelphia" "Philly"       "Phila"
```

or

```
# "Phil" followed by something that is not whitespace
grep("Phil[^\\s]", dataText, value=TRUE)
```

```
[1] "Philadelphia" "Philly"       "Phila"
```

10. Find capitalized words

```
grep("\\b[A-Z]", dataText, value=TRUE)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "Kendall Roy"        "Roman Roy"
 [7] "Siobhan Roy"        "Roy Wood, Jr."      "Royal Caribbean"
[10] "Casino Royale"      "Philadelphia"       "Philly"
[13] "Phila"              "Dr. Phil"
```

11. Find the addresses

```
grep("[0-9]+[A-Z]? [A-Za-z ]+ (Drive|Walk|Point|St)", dataText, value=TRUE)
```

```
[1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
[4] "221B Baker St."
```

12. Find a shorter way to get phone numbers using `?`

```
grep("\\(?[0-9]{3}(\\) |-| )?[0-9]{3}(-| )?[0-9]{4}", dataText, value=TRUE)
```

```
[1] "(215) 573-9097" "215-573-9097"   "2155739097"
```

13. Remove all the commas

```
gsub(",", "", dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phil"
[25] "$23456"             "$10000"             "$60000"
```

14. Add commas to these numbers `c("9453","2332","4645","1234")`.

```
gsub("([0-9])([0-9]{3})", "\\1,\\2", c("9453","2332","4645","1234"))
```

```
[1] "9,453" "2,332" "4,645" "1,234"
```

or

```
gsub("^([0-9])", "\\1,", c("9453","2332","4645","1234"))
```

```
[1] "9,453" "2,332" "4,645" "1,234"
```

15. Change Dr. Phil to Dr. Phillip

```
gsub("Phil$","Phillip",dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phillip"
[25] "$23456"             "$10000"             "$60,000"
```

16. Spell out Philadelphia

```
gsub("Phil(adelphia|ly|a)", "Philadelphia", dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philadelphia"       "Philadelphia"       "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

or

```
gsub("Phil\\B\\w*", "Philadelphia", dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "(215) 573-9097"
[10] "215-573-9097"       "2155739097"         "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philadelphia"       "Philadelphia"       "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

17. Keep just the first word or first number

```
gsub("^([^ ]+) .*$", "\\1", dataText)
```

```
 [1] "3718"         "4"            "10880"        "221B"         "19104-6286"
 [6] "20015"        "90291"        "90210"        "(215)"        "215-573-
9097"
[11] "2155739097"   "Kendall"      "Roman"        "Siobhan"      "Roy"
[16] "Royal"        "Casino"       "two"          "2"            "twins"
[21] "Philadelphia" "Philly"       "Phila"        "Dr."          "$23456"
[26] "$10000"       "$60,000"
```

or

```
gsub(" .*", "", dataText)
```

```
 [1] "3718"         "4"            "10880"        "221B"         "19104-6286"
 [6] "20015"        "90291"        "90210"        "(215)"        "215-573-
9097"
[11] "2155739097"   "Kendall"      "Roman"        "Siobhan"      "Roy"
[16] "Royal"        "Casino"       "two"          "2"            "twins"
[21] "Philadelphia" "Philly"       "Phila"        "Dr."          "$23456"
[26] "$10000"       "$60,000"
```

18. Keep only area codes of phone numbers

```
gsub("^\\(?([0-9]{3})(\\) |-| )?[0-9]{3}-?[0-9]{4}","\\1",dataText)
```

```
 [1] "3718 Locust Walk"   "4 Privet Drive"     "10880 Malibu Point"
 [4] "221B Baker St."     "19104-6286"         "20015"
 [7] "90291"              "90210"              "215"
[10] "215"                "215"                "Kendall Roy"
[13] "Roman Roy"          "Siobhan Roy"        "Roy Wood, Jr."
[16] "Royal Caribbean"    "Casino Royale"      "two children"
[19] "2 children"         "twins"              "Philadelphia"
[22] "Philly"             "Phila"              "Dr. Phil"
[25] "$23456"             "$10000"             "$60,000"
```

19. We were told "i before e except after c, or when sounded like a as in neighbor or weigh". Is that true?

```
grep("(^|[^c])ei", words, value=TRUE)
```

```
 [1] "their"        "being"        "either"       "neither"      "seeing"
 [6] "eight"        "foreign"      "weight"       "seized"       "height"
[11] "reign"        "sovereign"    "beings"       "eighteen"     "wherein"
[16] "veil"         "therein"      "leisure"      "seize"        "neighborhood"
[21] "heir"         "theirs"       "neighbors"    "veins"        "heights"
[26] "neighbour"    "eighty"       "neighbouring" "weighed"      "neighbor"
[31] "ein"          "reigned"      "sovereignty"  "foreigners"   "neighboring"
[36] "reins"        "eighteenth"   "seizing"      "weigh"        "vein"
[41] "Keith"        "sovereigns"   "eighth"       "leisurely"    "foreigner"
[46] "weird"        "deity"        "weighing"     "freight"      "deities"
[51] "rein"         "reigns"       "weighty"      "Raleigh"      "albeit"
[56] "heightened"   "Seine"        "well-being"   "forfeit"      "Marseilles"
[61] "twenty-eight" "herein"
```

```
grep("cie", words, value=TRUE)
```

```
 [1] "ancient"       "society"       "sufficient"    "species"
 [5] "science"       "conscience"    "sufficiently"  "scientific"
 [9] "fancied"       "fancies"       "efficient"     "efficiency"
[13] "tendencies"    "conscientious" "insufficient"  "deficient"
[17] "deficiency"
```

There are many words with "ei" either at the beginning or following a letter other than "c". There are also many words with "ie" immediately following "c".

20. Find words with punctuation

```
grep("[^a-zA-Z0-9]", words, value=TRUE)
```

```
  [1] "I'll"           "can't"          "I've"           "didn't"
  [5] "an'"            "won't"          "man's"          "c."
  [9] "that's"         "etc."           "o'"             "I'd"
 [13] "v."             "father's"       "wouldn't"       "he's"
 [17] "couldn't"       "isn't"          "'em"            "king's"
 [21] "there's"        "ain't"          "you'll"         "god's"
 [25] "mother+'s"      "wasn't"         "doesn't"        "haven't"
 [29] "you've"         "woman's"        "we'll"          "she's"
 [33] "'tis"           "you'd"          "what's"         "cf."
 [37] "hadn't"         "th'"            "pp."            "rest."
 [41] "o'er"           "u.s."           "he'd"           "they're"
 [45] "we're"          "day's"          "girl's"         "he'll"
 [49] "shouldn't"      "husband's"      "twenty-five"    "men's"
 [53] "other's"        "i.e."           "brother's"      "wife's"
 [57] "we've"          "well-known"     "lady's"         "twenty-four"
 [61] "ma'am"          "enemy's"        "imp."           "hasn't"
 [65] "majesty's"      "viz."           "mustn't"        "old-fashioned"
 [69] "friend's"       "so-called"      "re-use"         "aren't"
 [73] "they'll"        "needn't"        "pub."           "middle-aged"
 [77] "she'd"          "t'"             "of."            "oe."
 [81] "prof."          "she'll"         "shan't"         "ne'er"
 [85] "as."            "they'd"         "we'd"           "here's"
 [89] "they've"        "adj."           "weren't"        "sq."
 [93] "gen."           "cong."          "col."           "brother-in-law"
 [97] "son-in-law"     "e.g."           "ch."            "e-mail"
[101] "twenty-one"     "twenty-two"     "it'll"          "ft."
[105] "self-control"   "anglo-saxon"    "ff."            "forty-five"
[109] "'ll"            "Icel."          "good-looking"   "p.m."
[113] "e'er"           "twenty-three"   "thirty-six"     "a.m."
[117] "father-in-law"  "e'en"           "seventy-five"   "'cause"
[121] "well-being"     "Mass."          "thirty-two"     "self-"
[125] "twenty-eight"   "twenty-six"     "sister-in-law"  "three-quarters"
[129] "'n'"
```

or

```
grep("[[:punct:]]", words, value=TRUE)
```

```
  [1] "I'll"           "can't"          "I've"           "didn't"
  [5] "an'"            "won't"          "man's"          "c."
  [9] "that's"         "etc."           "o'"             "I'd"
 [13] "v."             "father's"       "wouldn't"       "he's"
 [17] "couldn't"       "isn't"          "'em"            "king's"
 [21] "there's"        "ain't"          "you'll"         "god's"
 [25] "mother+'s"      "wasn't"         "doesn't"        "haven't"
 [29] "you've"         "woman's"        "we'll"          "she's"
 [33] "'tis"           "you'd"          "what's"         "cf."
 [37] "hadn't"         "th'"            "pp."            "rest."
 [41] "o'er"           "u.s."           "he'd"           "they're"
 [45] "we're"          "day's"          "girl's"         "he'll"
 [49] "shouldn't"      "husband's"      "twenty-five"    "men's"
 [53] "other's"        "i.e."           "brother's"      "wife's"
 [57] "we've"          "well-known"     "lady's"         "twenty-four"
 [61] "ma'am"          "enemy's"        "imp."           "hasn't"
 [65] "majesty's"      "viz."           "mustn't"        "old-fashioned"
 [69] "friend's"       "so-called"      "re-use"         "aren't"
 [73] "they'll"        "needn't"        "pub."           "middle-aged"
 [77] "she'd"          "t'"             "of."            "oe."
 [81] "prof."          "she'll"         "shan't"         "ne'er"
 [85] "as."            "they'd"         "we'd"           "here's"
 [89] "they've"        "adj."           "weren't"        "sq."
 [93] "gen."           "cong."          "col."           "brother-in-law"
 [97] "son-in-law"     "e.g."           "ch."            "e-mail"
[101] "twenty-one"     "twenty-two"     "it'll"          "ft."
[105] "self-control"   "anglo-saxon"    "ff."            "forty-five"
[109] "'ll"            "Icel."          "good-looking"   "p.m."
[113] "e'er"           "twenty-three"   "thirty-six"     "a.m."
[117] "father-in-law"  "e'en"           "seventy-five"   "'cause"
[121] "well-being"     "Mass."          "thirty-two"     "self-"
[125] "twenty-eight"   "twenty-six"     "sister-in-law"  "three-quarters"
[129] "'n'"
```

`[[:punct:]]` is a character class containing punctuation characters. or

```
grep("\\W", words, value=TRUE)
```

```
  [1] "I'll"           "can't"          "I've"           "didn't"
```

```
  [5] "an'"            "won't"          "man's"          "c."
  [9] "that's"         "etc."           "o'"             "I'd"
 [13] "v."             "father's"       "wouldn't"       "he's"
 [17] "couldn't"       "isn't"          "'em"            "king's"
 [21] "there's"        "ain't"          "you'll"         "god's"
 [25] "mother+'s"      "wasn't"         "doesn't"        "haven't"
 [29] "you've"         "woman's"        "we'll"          "she's"
 [33] "'tis"           "you'd"          "what's"         "cf."
 [37] "hadn't"         "th'"            "pp."            "rest."
 [41] "o'er"           "u.s."           "he'd"           "they're"
 [45] "we're"          "day's"          "girl's"         "he'll"
 [49] "shouldn't"      "husband's"      "twenty-five"    "men's"
 [53] "other's"        "i.e."           "brother's"      "wife's"
 [57] "we've"          "well-known"     "lady's"         "twenty-four"
 [61] "ma'am"          "enemy's"        "imp."           "hasn't"
 [65] "majesty's"      "viz."           "mustn't"        "old-fashioned"
 [69] "friend's"       "so-called"      "re-use"         "aren't"
 [73] "they'll"        "needn't"        "pub."           "middle-aged"
 [77] "she'd"          "t'"             "of."            "oe."
 [81] "prof."          "she'll"         "shan't"         "ne'er"
 [85] "as."            "they'd"         "we'd"           "here's"
 [89] "they've"        "adj."           "weren't"        "sq."
 [93] "gen."           "cong."          "col."           "brother-in-law"
 [97] "son-in-law"     "e.g."           "ch."            "e-mail"
[101] "twenty-one"     "twenty-two"     "it'll"          "ft."
[105] "self-control"   "anglo-saxon"    "ff."            "forty-five"
[109] "'ll"            "Icel."          "good-looking"   "p.m."
[113] "e'er"           "twenty-three"   "thirty-six"     "a.m."
[117] "father-in-law"  "e'en"           "seventy-five"   "'cause"
[121] "well-being"     "Mass."          "thirty-two"     "self-"
[125] "twenty-eight"   "twenty-six"     "sister-in-law"  "three-quarters"
[129] "'n'"
```

\W matches any character that is not a letter, number, or underscore.

21. Find words that do not have aeiou in the first four characters

```
grep("^[^aeiouAEIOU]{4}", words, value=TRUE)
```

```
 [1] "system"        "physical"      "style"         "sympathy"
 [5] "mysterious"    "mystery"       "physician"     "thyself"
 [9] "sympathetic"   "mysteries"     "systems"       "Dryden"
```

```
[13] "symptoms"      "symbol"        "crystal"       "10th"
[17] "hymn"          "mystic"        "sympathies"    "Sydney"
[21] "12th"          "syllable"      "11th"          "rhyme"
[25] "symbols"       "physically"    "rhythm"        "systematic"
[29] "psychology"    "psychological" "p.m."          "mystical"
[33] "mythology"     "myth"          "syllables"     "hysterical"
```